

M7 TEORIA

MÓDULO 7 — Proyecto global: API REST de gestión de becas

Documento de diseño del sistema. La construcción paso a paso se realiza en `PRACTICA.md`.

Punto de partida de M7: el proyecto es nuevo e independiente. La preparación inicial crea dentro del propio módulo el soporte mínimo de Alumno, Usuario/Rol, JWT y errores que necesita el sistema completo.

♦ Punto 7.1 - Diseño de una API REST completa

🎯 Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Definir el alcance y los recursos de una API REST realista.
2. Modelar el dominio con entidades, relaciones y constraints.
3. Diseñar los endpoints con URLs, métodos y códigos de estado coherentes.
4. Identificar los DTOs necesarios y su propósito.
5. Definir las reglas de negocio y los roles de seguridad.
6. Documentar el diseño en un formato reutilizable.
7. Justificar cada decisión de diseño.

🧠 FASE DE DISEÑO

🕒 BLOQUE 1 - Alcance del proyecto

🎯 Punto T1.1 - Qué vamos a construir

Vamos a construir una API REST para la gestión de solicitudes de becas del Ministerio de Educación. Es un proyecto realista que cubre los casos de uso más comunes de una administración pública:

- Un ciudadano (o un funcionario en su nombre) consulta las becas disponibles.
- Un ciudadano crea una solicitud de beca para un alumno.
- Un ciudadano adjunta documentos a su solicitud.
- Un ciudadano consulta el estado de sus solicitudes.
- Un gestor revisa las solicitudes pendientes.
- Un gestor cambia el estado de una solicitud (aprobar, denegar, solicitar documentación).
- Un administrador gestiona las becas y los usuarios.

El proyecto no pretende ser un sistema completo de gestión de becas (eso requeriría meses de trabajo), sino un ejercicio integrador que aplica todo lo aprendido en los módulos anteriores. Cada decisión de diseño está pensada para que el alumno practique un concepto concreto: relaciones JPA, validaciones, seguridad, manejo de errores, etc.

Duración del proyecto: los 7 puntos del Módulo 7. Cada punto construye sobre el anterior. Al final, tendremos una API funcional con seguridad, tests y documentación.

Punto T1.2 - Actores del sistema

Antes de diseñar nada, hay que identificar quién va a usar la API y qué puede hacer. Los actores son:

Ciudadano: una persona que accede a la API para solicitar becas y consultar el estado de sus solicitudes. Se autentica con usuario y contraseña. En esta edición docente no se modela una relación Usuario → Alumno, por lo que la API no puede verificar ownership individual; las consultas de solicitudes se autorizan por rol y autenticación.

Gestor: un funcionario del Ministerio que revisa solicitudes y cambia su estado. Puede ver todas las solicitudes, pero solo modificar el estado y añadir observaciones. No puede crear solicitudes ni eliminarlas.

Administrador: un usuario con permisos totales. Puede gestionar las becas (crear, modificar, eliminar tipos de beca), gestionar los usuarios y acceder a cualquier recurso.

Sistema externo (opcional): otro sistema del Ministerio que consulta el estado de las solicitudes a través de la API. Se autentica con un token de servicio. No lo implementaremos en este curso, pero conviene tenerlo en cuenta para el diseño.

Regla: cada endpoint debe tener claro qué actor puede acceder y qué operaciones puede hacer. Si no, corremos el riesgo de dejar endpoints abiertos o de cerrar demasiado.

Punto T1.3 - Casos de uso principales

Los casos de uso son las operaciones concretas que los actores realizan sobre el sistema. Los agrupamos por actor:

Ciudadano: - Registrarse en el sistema. - Iniciar sesión y obtener un token. - Consultar las becas disponibles (público). - Crear una solicitud de beca para un alumno. - Consultar solicitudes. En un sistema real se limitarían a las propias; la edición docente no modela ownership Usuario → Alumno. - Consultar una solicitud concreta. - Adjuntar un documento a una solicitud. - Eliminar un documento de una solicitud (si la solicitud no está resuelta). - Cancelar una solicitud (si no está resuelta).

Gestor:

- Consultar todas las solicitudes (con filtros).
- Consultar una solicitud concreta.
- Cambiar el estado de una solicitud (APROBADA, DENEGADA, SOLICITUD_DOCUMENTACION).
- Añadir observaciones a una solicitud.

Administrador:

- Todo lo que puede hacer el gestor.
- Crear, modificar y eliminar tipos de beca.
- Gestionar usuarios (listar, consultar, cambiar roles, desactivar).

Regla: cada caso de uso se traduce en un endpoint o en una operación dentro de un endpoint. El diseño debe cubrir todos los casos de uso identificados.

BLOQUE 2 - Modelo de dominio

Punto T2.1 - Las entidades

El modelo de dominio tiene cinco entidades principales:

Beca: representa un tipo de beca (Beca General, Ayuda de Libros, Beca de Comedor...). Tiene un nombre, una descripción, un importe máximo, un año y un estado (activa/inactiva).

Alumno: representa a un alumno y se crea dentro de PREP como soporte inicial del proyecto. Tiene nombre, apellidos, DNI, fecha de nacimiento y curso. A partir de ese soporte, M7 amplía el modelo cuando lo requiere.

SolicitudBeca: representa una solicitud de beca que un alumno hace. Es el recurso central. Tiene un identificador, el alumno que la solicita, la beca solicitada, el estado, la fecha de solicitud, la fecha de resolución (si está resuelta), el importe concedido y las observaciones.

Documento: representa un documento adjunto a una solicitud (DNI, notas, certificado de renta...). Tiene un nombre, un tipo (PDF, imagen...), un tamaño y el contenido (opcionalmente).

Usuario: representa a un usuario del sistema y se crea dentro de PREP como soporte inicial de autenticación y autorización. Tiene username, password, email y roles.

Decisión de diseño: no creamos una entidad Ciudadano separada. El ciudadano es un Usuario con un rol concreto. El alumno es una entidad independiente. Un usuario puede solicitar becas para varios alumnos (sus hijos, por ejemplo), pero en este curso simplificamos y asumimos que cada solicitud pertenece a un alumno.

Punto T2.2 - Las relaciones

Las relaciones entre entidades son:

SolicitudBeca → Alumno: muchas solicitudes pertenecen a un alumno, pero un alumno puede tener varias solicitudes (una por año, una por tipo de beca). Es una relación @ManyToOne desde SolicitudBeca hacia Alumno.

SolicitudBeca → Beca: muchas solicitudes se hacen para una beca concreta, pero una beca puede tener muchas solicitudes. Es una relación @ManyToOne desde SolicitudBeca hacia Beca.

SolicitudBeca → Documento: una solicitud tiene varios documentos, y un documento pertenece a una solicitud. Es una relación @OneToMany desde SolicitudBeca hacia Documento, con mappedBy en Documento.

Usuario → Rol: relación @ManyToMany. La edición autónoma la crea antes de comenzar la implementación del dominio de becas.

SolicitudBeca → Usuario: opcionalmente, para saber quién creó la solicitud. En este curso no lo incluimos para simplificar, pero conviene saberlo.

Decisión de diseño: no creamos una relación directa entre Usuario y SolicitudBeca ni una relación Usuario → Alumno. Por tanto, el sistema no puede demostrar que una solicitud pertenece al usuario autenticado. La edición docente aplica autorización por rol y permite que los usuarios autenticados consulten solicitudes. En un sistema real se añadiría la relación Usuario → Alumno y se aplicaría ownership contextual.

Nota sobre la visibilidad de solicitudes. En un sistema real, un ciudadano solo vería sus propias solicitudes, lo que requeriría una relación Usuario → Alumno para saber qué alumnos puede gestionar cada ciudadano. En este curso simplificamos: cualquier usuario autenticado puede ver todas las solicitudes. La autorización contextual (que un ciudadano solo vea las suyas) se deja como ejercicio para el alumno que quiera profundizar. Esta simplificación es coherente con el objetivo del curso: practicar todas las capas de una API REST sin complicar el modelo de dominio.

Punto T2.3 - Los campos y constraints

Veamos los campos de cada entidad con sus constraints:

Beca:

- id (Long, PK, autoincrement).
- codigo (String, único, obligatorio, máximo 20 caracteres; incorporado por el reto 7.2.12).
- nombre (String, único, no nulo, 100 caracteres).
- descripcion (String, 500 caracteres).
- importeMaximo (BigDecimal, no nulo, positivo).
- anio (Integer, no nulo).
- activa (Boolean, no nulo, por defecto true).

SolicitudBeca:

- id (Long, PK, autoincrement).
- alumno (@ManyToOne, no nulo).
- beca (@ManyToOne, no nulo).
- estado (Enum, no nulo). Valores: BORRADOR, ENVIADA, EN_REVISION, SOLICITUD_DOCUMENTACION, APROBADA, DENEGADA, CANCELADA.
- fechaSolicitud (LocalDateTime, no nulo).
- fechaResolucion (LocalDateTime, puede ser nulo).
- importeConcedido (BigDecimal, puede ser nulo).
- observaciones (String, 1000 caracteres).
- documentos (@OneToMany, lista).

Documento:

- id (Long, PK, autoincrement).
- solicitud (@ManyToOne, no nulo).
- nombre (String, no nulo, 255 caracteres).
- tipo (String, no nulo, 50 caracteres).
- tamano (Long, no nulo, positivo).
- contenido (@Lob, byte[], opcional).
- fechaSubida (LocalDateTime, no nulo).

Alumno (preparado al inicio de la edición autónoma): id, nombre, apellidos, dni, fechaNacimiento, curso.

Usuario (preparado al inicio de la edición autónoma): id, username, password, email, activo, roles.

Decisión de diseño: usamos BigDecimal para importes (nunca Double). Usamos LocalDateTime para fechas con hora. El enum EstadoSolicitud se guarda como STRING (no ORDINAL).

BLOQUE 3 - Endpoints de la API

Punto T3.1 - Convenciones de URLs

Aplicamos las convenciones que ya conocemos:

- Prefijo: /api/v1/.
- Recursos en plural: /becas, /solicitudes, /alumnos, /usuarios.
- Minúsculas y guiones: /solicitudes-beca, /tipos-beca.
- Jerarquía para sub-recursos: /solicitudes/{id}/documentos.
- Path parameters para IDs: /solicitudes/123.
- Query parameters para filtros: /solicitudes?estado=ENVIADA.

Decisión de diseño: el recurso se llama solicitudes (no solicitudes-beca) porque es el recurso principal y no hay ambigüedad. Si en el futuro hubiera otro tipo de solicitudes, se renombraría.

Punto T3.2 - Tabla de endpoints

Esta es la tabla completa de endpoints de la API:

Método	URL	Descripción	Actor	Código éxito
POST	/api/v1/auth/registro	Registrar usuario	Público	201
POST	/api/v1/auth/login	Iniciar sesión	Público	200
POST	/api/v1/auth/refresh	Renovar token	Público	200
GET	/api/v1/becas	Listar becas activas	Público	200
GET	/api/v1/becas/{id}	Consultar beca	Público	200
POST	/api/v1/becas	Crear beca	ADMIN	201
PUT	/api/v1/becas/{id}	Actualizar beca	ADMIN	200
DELETE	/api/v1/becas/{id}	Eliminar beca	ADMIN	204
GET	/api/v1/alumnos	Listar alumnos	Autenticado	200
GET	/api/v1/alumnos/{id}	Consultar alumno	Autenticado	200
POST	/api/v1/alumnos	Crear alumno	CIUDADANO	201
PUT	/api/v1/alumnos/{id}	Actualizar alumno	CIUDADANO	200
DELETE	/api/v1/alumnos/{id}	Eliminar alumno	ADMIN	204
GET	/api/v1/solicitudes	Listar solicitudes	Autenticado	200
GET	/api/v1/solicitudes/{id}	Consultar solicitud	Autenticado	200
POST	/api/v1/solicitudes	Crear solicitud	CIUDADANO	201
PATCH	/api/v1/solicitudes/{id}/estado	Cambiar estado	GESTOR	200
DELETE	/api/v1/solicitudes/{id}	Cancelar solicitud	CIUDADANO	204
GET	/api/v1/solicitudes/{id}/documentos	Listar documentos	Autenticado	200
POST	/api/v1/solicitudes/{id}/documentos	Añadir documento	CIUDADANO	201
DELETE	/api/v1/solicitudes/{id}/documentos/{docId}	Eliminar documento	CIUDADANO	204
GET	/api/v1/usuarios	Listar usuarios	ADMIN	200
GET	/api/v1/usuarios/{id}	Consultar usuario	ADMIN	200
PATCH	/api/v1/usuarios/{id}/roles	Cambiar roles	ADMIN	200

PATCH /api/v1/usuarios/{id}/roles Cambiar roles ADMIN 200 Nota sobre la actualización de solicitudes. No incluimos un endpoint PUT /solicitudes/{id} para actualizar los datos de una solicitud. En el flujo de negocio, una vez creada la solicitud, solo se puede cambiar el estado (PATCH /solicitudes/{id}/estado) o cancelarla (DELETE /solicitudes/{id}). Actualizar los datos de una solicitud ya enviada no tiene sentido: si el ciudadano se equivocó, cancela y crea una nueva.

Decisión de diseño: algunos endpoints son públicos (auth, listar becas). Otros requieren autenticación (alumnos, solicitudes). Otros requieren rol específico (gestor, admin). La columna "Actor" indica el mínimo rol necesario.

🎯 Punto T3.3 - Filtros y paginación

Los endpoints de listado aceptan filtros y paginación:

GET /api/v1/solicitudes:

- estado (opcional): filtrar por estado.
- anio (opcional): filtrar por año de solicitud.
- becaId (opcional): filtrar por tipo de beca.
- alumnoId (opcional): filtrar por alumno.
- page (por defecto 0): número de página.
- size (por defecto 20): tamaño de página.
- sort (por defecto fechaSolicitud,desc): ordenación.

GET /api/v1/becas: - anio (opcional): filtrar por año. - activa (opcional): filtrar por estado activa/inactiva. - page, size, sort.

GET /api/v1/alumnos:

- curso (opcional): filtrar por curso.
- page, size, sort.

Decisión de diseño: los filtros van siempre en query parameters, nunca en la ruta. La paginación sigue la convención page/size/sort.

🔒 BLOQUE 4 - DTOs y validaciones

🎯 Punto T4.1 - DTOs de entrada

Para cada operación de creación o actualización, definimos un DTO de entrada con validaciones:

BecaRequestDTO:

- nombre (String, @NotBlank, @Size(max=100)).
- descripcion (String, @Size(max=500)).
- importeMaximo (BigDecimal, @NotNull, @PositiveOrZero).
- anio (Integer, @NotNull, @Min(2020), @Max(2100)).

AlumnoRequestDTO: se prepara al inicio de la edición autónoma.

SolicitudRequestDTO:

- alumnoId (Long, @NotNull).
- becaId (Long, @NotNull).
- observaciones (String, @Size(max=1000)).

CambioEstadoRequestDTO:

- estado (String, @NotBlank). Valores válidos: EN_REVISION, SOLICITUD_DOCUMENTACION, APROBADA, DENEGADA.
- observaciones (String, @Size(max=1000)).
- importeConcedido (BigDecimal, @PositiveOrZero). Obligatorio si el estado es APROBADA.

DocumentoRequestDTO: (se envía como multipart/form-data).

- nombre (String, @NotBlank).
- tipo (String, @NotBlank).
- contenido (MultipartFile, @NotNull).

Decisión de diseño: los DTOs de entrada solo incluyen los campos que el cliente puede enviar. El id, las fechas de auditoría y los estados iniciales los gestiona el servidor.

🎯 Punto T4.2 - DTOs de salida

Para cada recurso, definimos un DTO de salida:

BecaResponseDTO: id, codigo, nombre, descripcion, importeMaximo, anio, activa.

AlumnoResponseDTO: id, nombre, apellidos, dni, fechaNacimiento, curso, edad.

SolicitudResponseDTO: id, alumno (resumen), beca (resumen), estado, fechaSolicitud, fechaResolucion, importeConcedido, observaciones, numeroDocumentos.

DocumentoResponseDTO: id, nombre, tipo, tamaño, fechaSubida.

UsuarioResponseDTO: id, username, email, roles, activo.

Decisión de diseño: los DTOs de salida pueden incluir resúmenes de entidades relacionadas (por ejemplo, alumno con solo nombre y apellidos). Evitamos devolver entidades completas anidadas para no sobrecargar la respuesta.

🎯 Punto T4.3 - Reglas de negocio

Las reglas de negocio del sistema son:

Sobre solicitudes:

- Un alumno no puede tener dos solicitudes activas para la misma beca en el mismo año.
- Solo se puede modificar una solicitud si está en estado BORRADOR o ENVIADA.
- Solo se puede cambiar el estado de una solicitud si está en ENVIADA o EN_REVISION.
- Una solicitud APROBADA no se puede modificar ni eliminar.
- El importeConcedido es obligatorio cuando el estado es APROBADA.
- El importeConcedido no puede superar el importeMaximo de la beca.
- Al eliminar una solicitud, se eliminan sus documentos (cascade).

Sobre becas:

- Una beca no se puede eliminar si tiene solicitudes asociadas (o se hace soft delete).
- El año de la beca debe ser el actual o el siguiente.

Sobre documentos:

- Un documento solo se puede añadir si la solicitud está en BORRADOR o ENVIADA.
- Un documento no puede superar los 10 MB.
- Los tipos permitidos son PDF, JPG y PNG.

Decisión de diseño: las reglas de negocio van en el servicio, no en el controlador ni en el repositorio. Se lanzan excepciones personalizadas (NegocioException, RecursoDuplicadoException, etc.) que el manejador global convierte en respuestas HTTP.

🔒 BLOQUE 5 - Seguridad y estrategia

🎯 Punto T5.1 - Roles y permisos

El diseño base parte de tres roles de dominio y el reto 7.6.12 incorpora un cuarto rol de consulta:

ROLE_CIUDADANO: puede crear solicitudes, consultar solicitudes, añadir documentos y cancelar solicitudes. La edición docente no implementa ownership Usuario → Alumno.

ROLE_GESTOR: puede ver las solicitudes, cambiar estados y añadir observaciones. No puede crear solicitudes ni gestionar becas.

ROLE_ADMIN: puede gestionar becas, usuarios y acceder a todos los recursos administrativos.

ROLE_CONSULTOR: incorporado en 7.6.12; puede realizar operaciones GET autorizadas sobre solicitudes, pero no cambios de estado ni operaciones de gestión.

Decisión de diseño: los roles se definen en una tabla `roles` creada por la preparación autónoma de M7. Los permisos se aplican con reglas de URL para las áreas generales y con `@PreAuthorize` para las operaciones específicas.

Punto T5.2 - Estrategia de seguridad

La estrategia de seguridad es:

- Autenticación con JWT. Login con usuario y contraseña, tokens con expiración corta, refresh tokens.
- Filtro JWT, creado en la preparación autónoma del propio M7.
- Stateless. No se crean sesiones.
- CORS configurado para el front-end.
- CSRF desactivado (API REST).
- HTTPS en producción.
- Tokens en variables de entorno.

Decisión de diseño: sólo `POST /api/v1/auth/registro`, `POST /api/v1/auth/login` y `POST /api/v1/auth/refresh` quedan en la allowlist pública de autenticación. Swagger/H2 se permiten únicamente en el contexto docente/de desarrollo y los GET de becas son públicos. El resto se protege por autenticación y roles. El reto 7.6.12 añade CONSULTOR para consultas GET de solicitudes.

Punto T5.3 - Manejo de errores

La estrategia de errores usa el mismo patrón didáctico trabajado anteriormente; en esta edición autónoma se crea dentro del propio M7:

- Excepciones personalizadas: `RecursoNoEncontradoException`, `RecursoDuplicadoException`, `OperacionNoPermitidaException`, `ValidacionNegocioException`.
- Respuestas estándar: `ErrorResponse` con timestamp, status, codigo, mensaje, path, detalles, errors.
- Manejador global: `@RestControllerAdvice` con un manejador por excepción.
- Errores de validación: `MethodArgumentNotValidException` con lista de errores.
- Errores de seguridad: `AuthenticationEntryPoint` y `AccessDeniedHandler` con JSON.

Decisión de diseño: todos los errores de la API siguen la misma estructura. El cliente escribe un solo manejador.

Resumen del diseño

Elemento Decisión

Recursos Beca, Alumno, SolicitudBeca, Documento, Usuario

`@ManyToOne (Solicitud→Alumno,`

`Relaciones Solicitud→Beca), @OneToMany (Solicitud→Documento), @ManyToMany (Usuario→Rol)`

Endpoints 24 endpoints base; 26 en el proyecto final tras 7.5.12 y 7.7.13

DTOs Entrada con validaciones, salida con resúmenes

Reglas de 12 reglas atómicas, en los servicios negocio

Roles CIUDADANO, GESTOR, ADMIN; CONSULTOR se incorpora en 7.6.12

Seguridad JWT, stateless, CORS, HTTPS

Errores Estructura estándar con código, mensaje, detalles

Testing Tests de integración con @SpringBootTest y MockMvc

VERIFICACIÓN DEL DISEÑO

♦ Punto 7.2 - Implementación de entidades

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Crear entidades JPA para el dominio de becas.
2. Modelar relaciones @ManyToOne y @OneToMany con mappedBy y @JoinColumn.
3. Aplicar constraints (nullable, unique, length, precision) según las reglas del dominio.
4. Configurar cascade y orphanRemoval según la naturaleza de cada relación.
5. Verificar el esquema generado por Hibernate.
6. Diagnosticar y resolver los errores más comunes al definir entidades con relaciones.

FASE DE DISEÑO

BLOQUE 1 - Qué entidades vamos a crear

En este punto creamos las tres entidades propias del dominio de becas: Beca, SolicitudBeca y Documento. Alumno y Usuario ya están disponibles dentro del propio M7 porque PREP los crea como soporte autónomo; desde 7.2 sólo se amplían cuando el dominio de becas lo necesita.

Beca es una entidad sencilla, sin relaciones salientes. Solo tiene campos descriptivos y de configuración. Es la entidad “catálogo”: existen unas pocas becas y muchas solicitudes apuntan a ellas. SolicitudBeca es la entidad central del dominio. Tiene dos relaciones @ManyToOne (hacia Alumno y hacia Beca) y una relación @OneToMany (hacia Documento). Es la que más reglas de negocio tiene asociadas.

Documento es una entidad dependiente de SolicitudBeca. No tiene sentido sin la solicitud a la que pertenece. Por eso la relación es @OneToMany con cascade y orphanRemoval.

Decisión de diseño: no creamos una entidad UsuarioSolicitud ni nada parecido. El control de qué solicitudes puede ver cada usuario se hace mediante las reglas de rol documentadas. En un sistema real, el ciudadano se limitaría a las solicitudes asociadas a sus alumnos mediante una relación Usuario → Alumno. Como M7 no define esa relación verificable, la edición docente no aplica ownership por usuario.

BLOQUE 2 - Relaciones con las entidades existentes

Las entidades nuevas se relacionan con las existentes así:

SolicitudBeca → Alumno: @ManyToOne desde SolicitudBeca. Muchas solicitudes pertenecen a un alumno. La columna de clave foránea (alumno_id) estará en la tabla solicitudes.

SolicitudBeca → Beca: @ManyToOne desde SolicitudBeca. Muchas solicitudes apuntan a una beca. La columna de clave foránea (beca_id) estará en la tabla solicitudes.

SolicitudBeca → Documento: @OneToMany desde SolicitudBeca, con mappedBy = “solicitud” en el lado de Documento. La columna de clave foránea (solicitud_id) estará en la tabla

documentos. Decisión de diseño: no añadimos ninguna relación inversa @OneToMany en Alumno ni en Beca. No las necesitamos para las operaciones del proyecto, y añadirlas complicaría el modelo sin aportar valor. Si en el futuro hiciera falta (por ejemplo, “listar todas las solicitudes de un alumno”), se pueden añadir con mappedBy sin romper nada.

🕒 BLOQUE 3 - Constraints y decisiones de campos

Repasamos los constraints de cada entidad según lo que decidimos en el diseño (punto 7.1):

Beca:

- nombre: @NotBlank, @Size(max=100), unique=true. No puede haber dos becas con el mismo nombre.
- importeMaximo: @NotNull, @PositiveOrZero, precision=10, scale=2. Big decimal para no perder precisión.
- anio: @NotNull, @Min(2020), @Max(2100).
- activa: @NotNull, por defecto true.

SolicitudBeca:

- estado: @Enumerated(EnumType.STRING), @Column(nullable=false, length=30). Guardado como string, no como ordinal.
- fechaSolicitud: @Column(nullable=false), se asigna en el servicio al crear.
- fechaResolucion: @Column sin nullable=false, porque solo se rellena al resolver.
- importeConcedido: @Column(precision=10, scale=2), sin nullable=false, porque solo se rellena al aprobar.

Documento:

- nombre: @NotBlank, @Size(max=255).
- tipo: @NotBlank, @Size(max=50).
- tamano: @NotNull, @PositiveOrZero.
- contenido: @Lob, opcional (podría guardarse en disco en lugar de en la BD, pero en este curso lo guardamos en la BD para simplificar).

♦ Punto 7.3 - Implementación de repositorios

🎯 Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Crear repositorios Spring Data JPA para las entidades del proyecto.
2. Definir métodos derivados según las necesidades del diseño.
3. Escribir consultas con @Query para filtros complejos.
4. Aplicar paginación y ordenación con Pageable.
5. Usar @EntityGraph para cargar relaciones y evitar el problema N+1.
6. Escribir consultas de agregación.
7. Diagnosticar y resolver los errores más comunes al definir repositorios.

🧠 FASE DE DISEÑO

🕒 BLOQUE 1 - Qué repositorios necesitamos

En este punto creamos tres repositorios nuevos, uno por cada entidad nueva del punto 7.2:

BecaRepository: gestiona las becas. Necesita operaciones CRUD básicas (heredadas de JpaRepository) y algunas consultas específicas:

- Buscar por nombre (para validar duplicados).
- Buscar por código (si añadimos el campo codigo en el reto).
- Listar becas activas de un año concreto.
- Listar con filtros combinados (activa + anio) y paginación.
- Paginación y ordenación para el listado.

SolicitudBecaRepository: es el repositorio más complejo del proyecto. Necesita:

- CRUD básico.
- Buscar por alumno.
- Buscar por beca.
- Buscar por estado.
- Buscar por rango de fechas.
- Combinaciones de filtros (alumno + beca + estado).
- Paginación y ordenación.
- Consultas con JOIN FETCH para evitar el problema N+1 al cargar alumno y beca.

DocumentoRepository: gestiona los documentos. Necesita: - CRUD básico. - Buscar documentos por solicitud. - Contar documentos por solicitud. - Eliminar documentos por solicitud (útil para cascade).

Decisión de diseño: los repositorios solo contienen operaciones de acceso a datos. Nada de lógica de negocio. Las validaciones y reglas van en los servicios (punto 7.4).

🕒 BLOQUE 2 - Estrategia de consultas

Para cada repositorio, decidimos qué tipo de consultas usar:

BecaRepository: la mayoría de las consultas son métodos derivados para filtros simples, y una consulta con @Query para el filtrado combinado con paginación.

SolicitudBecaRepository: mezcla de métodos derivados y @Query. Los métodos derivados para filtros simples; @Query con JOIN FETCH para los listados que necesitan cargar relaciones.

DocumentoRepository: métodos derivados. La mayoría son findBySolicitudId y countBySolicitudId.

Decisión de diseño: no usamos Specifications en este proyecto. Los filtros de solicitudes se pueden combinar, pero no son tan dinámicos como para necesitar Specifications. Con métodos derivados y un par de @Query cubrimos los casos.

Si en el futuro se necesitaran filtros más complejos (por ejemplo, búsqueda por texto libre en observaciones), se pueden añadir Specifications sin romper lo existente.

🕒 BLOQUE 3 - Paginación y ordenación

Los listados de becas y solicitudes se pagan. Spring Data JPA ofrece Pageable y Page para esto (los vimos en el Módulo 4).

Decisión de diseño:

- BecaRepository: método buscarConFiltros(activa, anio, pageable) que devuelve Page.
- SolicitudBecaRepository: método buscarConFiltros(estado, alumnoId, becaId, pageable) que devuelve Page.
- Ordenación por defecto: por id ascendente. Si el cliente no especifica, se usa esta.

🕒 BLOQUE 4 - Carga de relaciones

El repositorio de solicitudes necesita cargar el alumno y la beca cuando se listan o consultan solicitudes. Sin esto, se produciría el problema N+1 (una consulta por cada solicitud para cargar el alumno, y otra por cada una para cargar la beca).

Decisión de diseño:

- Para consultas individuales (findById): usamos @Query con JOIN FETCH para cargar las relaciones en una sola consulta.
- Para listados con filtros: usamos @Query con JOIN FETCH y filtros condicionales.
- Para listados simples: usamos @EntityGraph para indicar qué relaciones cargar.

♦ Punto 7.4 - Implementación de servicios

🎯 Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Crear los DTOs de entrada y salida del proyecto.
2. Implementar servicios con lógica de negocio y transacciones.
3. Aplicar las reglas de negocio del diseño con excepciones personalizadas.
4. Transformar entre entidades y DTOs dentro del servicio.
5. Manejar relaciones LAZY dentro de la transacción.
6. Validar transiciones de estado.
7. Escribir tests unitarios de los servicios.
8. Diagnosticar y resolver los errores más comunes al implementar servicios.

🧠 FASE DE DISEÑO

🕒 BLOQUE 1 - Qué servicios necesitamos

En este punto implementamos tres servicios, uno por cada recurso con lógica de negocio: BecaService: gestiona el catálogo de becas. Sus operaciones son CRUD simples, con validaciones básicas (nombre único, no eliminar becas con solicitudes asociadas).

SolicitudService: es el servicio central del proyecto. Contiene la mayoría de las reglas de negocio. Valida duplicados, controla las transiciones de estado, calcula importes, coordina con el repositorio de documentos.

DocumentoService: gestiona los documentos adjuntos a las solicitudes. Valida que la solicitud esté en un estado que permita modificaciones, y que el documento cumpla los requisitos (tamaño, tipo).

Decisión de diseño: cada servicio tiene una responsabilidad clara. BecaService no sabe nada de solicitudes. SolicitudService conoce las solicitudes pero delega la gestión de documentos en DocumentoService. DocumentoService conoce los documentos pero consulta al repositorio de solicitudes para validar el estado.

🕒 BLOQUE 2 - DTOs que necesitamos

Antes de implementar los servicios, necesitamos definir los DTOs. Recordemos el diseño del punto 7.1:

DTOs de entrada:

- BecaRequestDTO: los datos que el cliente envía para crear o actualizar una beca.
- SolicitudRequestDTO: los datos para crear una solicitud.
- CambioEstadoRequestDTO: los datos para cambiar el estado de una solicitud.

DTOs de salida: - BecaResponseDTO: los datos que se devuelven de una beca. - SolicitudResponseDTO: los datos que se devuelven de una solicitud, con resúmenes del alumno y la beca. - DocumentoResponseDTO: los datos que se devuelven de un documento (sin el contenido binario).

DTOs auxiliares:

- AlumnoResumenDTO: resumen del alumno para anidar en SolicitudResponseDTO.
- BecaResumenDTO: resumen de la beca para anidar en SolicitudResponseDTO.

Decisión de diseño: los DTOs de salida incluyen resúmenes de las entidades relacionadas (no las entidades completas). Eso evita respuestas gigantes y problemas de serialización con relaciones LAZY.

🕒 BLOQUE 3 - Reglas de negocio

Repasamos las reglas del diseño (punto 7.1) y dónde se implementan:

Regla Servicio

Un alumno no puede tener dos solicitudes SolicitudService.crear activas para la misma beca en el mismo año Regla Servicio

Solo se puede modificar una solicitud si está en SolicitudService.actualizar BORRADOR o ENVIADA

Solo se puede cambiar el estado si está en SolicitudService.cambiarEstado ENVIADA o EN_REVISION

Una solicitud APROBADA no se puede modificar SolicitudService.actualizar/eliminar ni eliminar

El importeConcedido es obligatorio si el estado SolicitudService.cambiarEstado es APROBADA

El importeConcedido no puede superar el SolicitudService.cambiarEstado importeMaximo de la beca

Las transiciones de estado están restringidas SolicitudService.cambiarEstado (con validarTransicion)

Al eliminar una solicitud, se eliminan sus cascade en la entidad documentos Regla Servicio

Una beca no se puede eliminar si tiene BecaService.eliminar solicitudes

Un documento solo se puede añadir si la DocumentoService.añadir solicitud está en BORRADOR o ENVIADA

Un documento no puede superar 10 MB. Tipos DocumentoService.añadir permitidos: PDF, JPG, PNG

Decisión de diseño: todas las reglas se implementan en el servicio, con excepciones personalizadas del Módulo 5. El controlador solo delega; el repositorio solo accede a datos.

🕒 BLOQUE 4 - Transformación DTO ↔ Entidad

Los servicios son los encargados de transformar entre entidades y DTOs. Los métodos de transformación son privados y se llaman desde los métodos públicos:

- toDTO(entidad): convierte una entidad a su DTO de salida.
- toEntity(dto): convierte un DTO de entrada a entidad. Para SolicitudResponseDTO, la transformación incluye los resúmenes del alumno y la beca. Como esas relaciones son

LAZY, la transformación debe hacerse dentro de la transacción, cuando la sesión de Hibernate está abierta. Eso ya lo sabemos del Módulo

1.

Decisión de diseño: los métodos toDTO y toEntity van en el servicio, no en los DTOs ni en las entidades. Cada servicio conoce los DTOs que le corresponden.

BLOQUE 5 - Transacciones

Los servicios usan @Transactional según lo aprendido en el Módulo 4.6:

- @Transactional(readOnly = true) en métodos de consulta.
- @Transactional en métodos que modifican datos.

Decisión de diseño: los servicios son la única capa con transacciones. Los controladores no tienen @Transactional. Los repositorios heredan las transacciones por defecto de Spring Data, pero las transacciones de negocio las define el servicio.

◆ Punto 7.5 - Implementación de controladores

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Implementar controladores REST que deleguen en los servicios.
2. Aplicar códigos de estado correctos en cada operación.
3. Construir respuestas con ResponseEntity y cabecera Location.
4. Combinar @Valid, @PathVariable, @RequestParam y @RequestBody.
5. Documentar los endpoints con OpenAPI.
6. Implementar el UsuarioController para la gestión de usuarios (solo ADMIN).
7. Verificar los controladores con MockMvc.
8. Diagnosticar y resolver los errores más comunes al implementar controladores.

FASE DE DISEÑO

BLOQUE 1 - Qué controladores necesitamos

En este punto creamos cuatro controladores, uno por cada recurso con endpoints públicos:

BecaController: expone el catálogo de becas. Tiene endpoints de listado (público), consulta (público), creación, actualización y eliminación (solo ADMIN).

SolicitudController: el controlador central del proyecto. Expone los endpoints de gestión de solicitudes: listar, consultar, crear, cambiar estado y cancelar.

DocumentoController: expone los documentos. Aunque los documentos son sub-recursos de las solicitudes, tienen sus propios endpoints para añadir, listar y eliminar.

UsuarioController: expone la gestión de usuarios. Solo ADMIN puede acceder. Permite listar, consultar y cambiar roles.

Decisión de diseño: separamos los documentos en su propio controlador en lugar de meterlos en SolicitudController. La razón es que los documentos tienen su propia lógica (subida de ficheros) y sus propios permisos. Meterlos en SolicitudController lo haría demasiado grande. El UsuarioController es necesario porque en el punto 7.1 se diseñan endpoints de gestión de usuarios. PREP proporciona la base de usuarios y 7.5 la amplía con las operaciones administrativas necesarias para M7.

🕒 BLOQUE 2 - Estructura de un controlador

Cada controlador sigue el mismo patrón:

- @RestController a nivel de clase.
- @RequestMapping("/api/v1/recurso") a nivel de clase.
- Inyección por constructor del servicio correspondiente.
- Métodos públicos para cada endpoint.
- ResponseEntity como tipo de retorno para controlar códigos y cabeceras.
- @Valid en los cuerpos de entrada.
- Anotaciones OpenAPI para documentar cada endpoint.

Decisión de diseño: los controladores no tienen lógica de negocio. Solo reciben la petición, delegan en el servicio y construyen la respuesta. Cualquier validación de negocio está en el servicio.

🕒 BLOQUE 3 - Endpoints de cada controlador

Repasamos los endpoints del diseño (punto 7.1) y los asignamos a cada controlador:

BecaController:

- GET /api/v1/becas → listar con filtros y paginación.
 - GET /api/v1/becas/{id} → consultar.
 - POST /api/v1/becas → crear (ADMIN).
 - PUT /api/v1/becas/{id} → actualizar (ADMIN).
 - DELETE /api/v1/becas/{id} → eliminar (ADMIN).
- SolicitudController:
- GET /api/v1/solicitudes → listar con filtros y paginación.
 - GET /api/v1/solicitudes/{id} → consultar.
 - POST /api/v1/solicitudes → crear (CIUDADANO).
 - PATCH /api/v1/solicitudes/{id}/estado → cambiar estado (GESTOR).
 - DELETE /api/v1/solicitudes/{id} → cancelar (CIUDADANO).

DocumentoController:

- GET /api/v1/solicitudes/{solicitudId}/documentos → listar documentos.
- POST /api/v1/solicitudes/{solicitudId}/documentos → añadir documento (CIUDADANO).
- DELETE /api/v1/solicitudes/{solicitudId}/documentos/{documentoId} → eliminar documento (CIUDADANO).

UsuarioController:

- GET /api/v1/usuarios → listar con paginación (ADMIN).
- GET /api/v1/usuarios/{id} → consultar (ADMIN).
- PATCH /api/v1/usuarios/{id}/roles → cambiar roles (ADMIN).

Decisión de diseño: los endpoints de documentos van bajo la ruta de la solicitud, porque los documentos son sub-recursos. La URL refleja la jerarquía: /solicitudes/{id}/documentos. Los endpoints de usuarios van bajo /usuarios y requieren ADMIN.

Nota sobre la actualización de solicitudes. No incluimos un endpoint PUT /solicitudes/{id}. En el flujo de negocio, una vez creada la solicitud, solo se puede cambiar el estado (PATCH /solicitudes/{id}/estado) o cancelarla (DELETE /solicitudes/{id}).

🕒 BLOQUE 4 - Seguridad de los endpoints

La seguridad se configura en SecurityConfig (punto 7.6), pero cada método del controlador tiene claro qué rol necesita. Los endpoints se protegen con:

- Reglas de URL en SecurityConfig para áreas generales (/api/v1/becas/, /api/v1/admin/).
- @PreAuthorize en los métodos para reglas específicas (crear solicitud, cambiar estado).

Decisión de diseño: en este punto (7.5) escribimos los controladores sin seguridad. En el punto 7.6 añadimos la seguridad. Así cada punto se centra en una cosa.

🕒 BLOQUE 5 - Documentación con OpenAPI

Cada endpoint se documenta con anotaciones de Springdoc:

- @Operation(summary = "...") → descripción breve del endpoint.
- @ApiResponse(responseCode = "...", description = "...") → respuestas posibles.
- @Parameter(description = "...") → descripción de parámetros.

Decisión de diseño: documentamos los endpoints principales. No hace falta documentar cada parámetro con detalle; lo esencial es que Swagger UI muestre qué hace cada endpoint y qué respuestas puede devolver.

♦ Punto 7.6 - Seguridad básica

🎯 Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Integrar Spring Security y JWT en el proyecto completo.
2. Definir roles específicos del dominio (CIUDADANO, GESTOR, ADMIN).
3. Configurar el SecurityFilterChain con reglas por endpoint.
4. Aplicar @PreAuthorize en las operaciones que lo requieren.
5. Configurar CORS en la cadena de filtros de seguridad.
6. Inicializar usuarios y roles con datos de ejemplo.
7. Verificar la seguridad con curl y con tests.
8. Diagnosticar y resolver los errores más comunes al integrar seguridad en un proyecto.

🧠 FASE DE DISEÑO

🕒 BLOQUE 1 - Qué necesitamos

SecurityConfig, JwtAuthenticationFilter, JwtService, Usuario, Rol, UsuarioDetailsService, JwtAuthenticationEntryPoint y JwtAccessDeniedHandler se crean en la preparación inicial del propio M7. En el punto 7.6 esa infraestructura se especializa con los permisos del dominio de becas. Lo que hay que hacer:

1. Añadir los roles de dominio CIUDADANO y GESTOR sobre la base preparada, y reservar CONSULTOR para el reto 7.6.12.
2. Actualizar el SecurityFilterChain con las reglas específicas de los endpoints del proyecto.
3. Configurar CORS en la cadena de filtros de seguridad.
4. Aplicar @PreAuthorize en las operaciones que necesitan autorización contextual.
5. Inicializar usuarios y roles con datos de ejemplo para poder probar.
6. Verificar que todo funciona con curl.

Decisión de diseño: reutilizamos dentro del propio M7 la infraestructura JWT creada en PREP-04..PREP-06 y la adaptamos al dominio de becas; M7 es técnicamente autónomo.

🕒 BLOQUE 2 - Los roles del dominio

Antes del reto 7.6.12, los roles disponibles en el proyecto son cuatro (incluido USER de soporte de autenticación):

ROLE_CIUDADANO: puede crear solicitudes, consultar solicitudes, añadir documentos, cancelar solicitudes. No puede cambiar estados ni gestionar becas.

ROLE_GESTOR: puede ver todas las solicitudes, cambiar estados, añadir observaciones. No puede crear solicitudes ni gestionar becas.

ROLE_ADMIN: puede hacer todo. Gestionar becas, usuarios y acceder a cualquier recurso.

ROLE_USER: rol genérico que se asigna por defecto al registrarse. En este proyecto, puede consultar becas, gestionar alumnos, pero no crear solicitudes. Es un rol “en pruebas” hasta que el administrador le asigne un rol concreto. Decisión de diseño: los roles se guardan en la tabla `roles`, creada en PREP dentro del propio M7. Se identifican por su nombre:

CIUDADANO, GESTOR, ADMIN, USER. En las authorities, se añade el prefijo `ROLE_` automáticamente al transformar.

🕒 BLOQUE 3 - Reglas de seguridad por endpoint

Repasamos los endpoints del proyecto (punto 7.1) y les asignamos reglas de seguridad:

Endpoint Método Regla Motivo

POST `/api/v1/auth/registro` POST `permitAll` Alta de usuario
 POST `/api/v1/auth/login` POST `permitAll` Autenticación
 POST `/api/v1/auth/refresh` POST `permitAll` Renovación de token

GET `/api/v1/becas` GET `permitAll` Público (catálogo de becas)

GET `/api/v1/becas/{id}` GET `permitAll` Público (detalle de beca)

POST `/api/v1/becas` POST `hasRole('ADMIN')` Solo admin puede crear becas

PUT `/api/v1/becas/{id}` PUT `hasRole('ADMIN')` Solo admin

DELETE `/api/v1/becas/{id}` DELETE `hasRole('ADMIN')` Solo admin
 Endpoint Método Regla Motivo

`/api/v1/solicitudes/**` Todos `authenticated` Requiere autenticación

POST `/api/v1/solicitudes` POST `hasRole('CIUDADANO')` Solo ciudadanos

PATCH `/api/v1/solicitudes/{id}/estado` PATCH `hasAnyRole('GESTOR','ADMIN')` Solo gestores o admins

DELETE `/api/v1/solicitudes/{id}` DELETE `hasRole('CIUDADANO')` Solo ciudadanos cancelan

POST `/api/v1/solicitudes/{id}/documentos` POST `hasRole('CIUDADANO')` Solo ciudadanos

DELETE `/api/v1/solicitudes/{id}/documentos/{docId}` DELETE `hasRole('CIUDADANO')` Solo ciudadanos

`/api/v1/usuarios/**` Todos `hasRole('ADMIN')` Solo admin

`/h2-console/**` Todos `permitAll` Solo desarrollo

`/swagger-ui/`, `/v3/api-docs/` Todos `permitAll` Documentación Endpoint Método Regla Motivo

Cualquier otra ruta — `authenticated` Por defecto, autenticado

Decisión de diseño: las reglas generales por área se configuran en el `SecurityFilterChain`. Las reglas contextuales (por ejemplo, “una operación requiere un rol concreto”) se aplican con `@PreAuthorize` en el controlador o en el servicio.

🕒 BLOQUE 4 - CORS en la cadena de filtros

Cuando se usa Spring Security, la configuración de CORS debe registrarse en la cadena de filtros de seguridad. Si solo se configura en WebMvcConfigurer, Spring Security bloquea las peticiones preflight OPTIONS con un 401, y el navegador no puede consumir la API.

Spring Security tiene su propia cadena de filtros y, por defecto, no aplica la configuración de CORS de Spring MVC. Para que la respete, hay que:

1. Declarar un bean CorsConfigurationSource con la configuración.
2. Habilitar CORS en la configuración de seguridad con http.cors().

Sin esta configuración, el preflight falla y el front-end no puede consumir la API. Es un error muy común y difícil de diagnosticar porque los tests con curl funcionan (no aplican SOP) pero el navegador no.

Decisión de diseño: configuramos CORS en el SecurityFilterChain con CorsConfigurationSource como bean. Es la forma correcta cuando se usa Spring Security.

🕒 BLOQUE 5 - Datos iniciales de seguridad

Para poder probar la seguridad, necesitamos usuarios con los cuatro roles:

- admin con rol ADMIN. Puede gestionar becas, usuarios y ver todo.
- gestor con rol GESTOR. Puede ver todas las solicitudes y cambiar estados.
- ciudadano con rol CIUDADANO. Puede crear y cancelar solicitudes.
- user con rol USER. Puede consultar becas y gestionar alumnos.

Decisión de diseño: los usuarios iniciales se crean en un CommandLineRunner con @Profile("dev"). En producción, los crea el administrador desde la API. El reto 7.6.12 añade CONSULTOR y verifica que puede consultar pero no modificar estados.

♦ Punto 7.7 - Pruebas finales del API

🎯 Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Verificar el proyecto completo con una batería de pruebas manuales.
2. Ejecutar todos los tests automatizados y analizar los resultados.
3. Revisar la documentación de OpenAPI y validar los contratos.
4. Preparar una colección de Postman lista para entregar.
5. Realizar una prueba de carga básica con ab.
6. Documentar el proyecto con un README completo.
7. Diagnosticar y resolver los errores más comunes antes de la entrega.

🧠 FASE DE DISEÑO

🕒 BLOQUE 1 - Qué vamos a verificar

El proyecto está terminado. Ahora toca verificar que funciona antes de darlo por finalizado. La verificación cubre cinco áreas:

Funcionalidad: se comprueba con curl y Postman que todos los endpoints respondan según el diseño.

Seguridad: se prueba con distintos usuarios y se verifica que los roles y los códigos de estado sean los esperados.

Tests automatizados: se ejecuta `./mvnw test` y se analizan los resultados.

Documentación: se revisa Swagger UI y se comprueba que OpenAPI refleje exclusivamente la API real.

Rendimiento básico: se realiza una prueba de carga ligera con `ab` y se observan tiempos y errores.

Decisión de diseño: esta verificación es manual y automatizada. Las pruebas manuales (`curl`, Postman) detectan problemas de integración. Los tests automatizados detectan regresiones. La documentación garantiza que los consumidores entiendan la API.

🕒 BLOQUE 2 - Escenarios de prueba

Vamos a definir los escenarios que hay que probar: Escenario 1 - Flujo completo del ciudadano:

1. Registrarse.
2. Login.
3. Consultar becas.
4. Crear una solicitud.
5. Añadir un documento.
6. Consultar la solicitud.
7. Cancelar la solicitud.

Escenario 2 - Flujo completo del gestor:

1. Login.
2. Listar solicitudes pendientes.
3. Cambiar el estado de una solicitud a `EN_REVISION`.
4. Cambiar el estado a `APROBADA` con importe.
5. Verificar que la solicitud está resuelta.

Escenario 3 - Flujo completo del administrador:

1. Login.
2. Crear una beca.
3. Listar usuarios.
4. Cambiar el rol de un usuario.
5. Eliminar una beca sin solicitudes.

Escenario 4 - Errores y seguridad:

1. Acceder a un endpoint protegido sin token → 401.
2. Acceder a un endpoint de admin con rol `CIUDADANO` → 403.
3. Crear una solicitud duplicada → 409.
4. Cambiar el estado de una solicitud aprobada → 409.
5. Subir un fichero demasiado grande → 400.

Escenario 5 - Documentación:

1. Abrir Swagger UI.
2. Verificar que todos los endpoints aparecen.
3. Probar un endpoint desde Swagger UI.

Decisión de diseño: los escenarios cubren los casos de éxito y los casos de error. Si todos pasan, el proyecto está listo.

🕒 BLOQUE 3 - Herramientas de verificación

Usaremos cuatro herramientas:

curl: para pruebas rápidas desde la terminal. Ya lo conocemos.

Postman: para organizar las pruebas en una colección. Es lo que se entrega al cliente.

JUnit + MockMvc: para los tests automatizados. Ya los tenemos. ab (Apache Benchmark): para la prueba de carga ligera. Se instala con el paquete apache2-utils, que está disponible en cualquier distribución Linux.

Decisión de diseño: no usamos herramientas pesadas como JMeter o Gatling. La prueba de carga es ligera: solo para verificar que la API no se cae bajo una carga moderada. Priorizamos ab porque viene con el sistema (vía apache2-utils) y no requiere instalar Go ni otras dependencias.

🕒 BLOQUE 4 - Qué se entrega

Al final del proyecto, se entrega:

- Código fuente: en el repositorio Git.
- README.md: con instrucciones de instalación, ejecución, endpoints y credenciales de prueba.
- Colección de Postman: con todas las peticiones organizadas.
- Documentación OpenAPI: accesible en /swagger-ui.html.
- Informe de tests: con el resultado de ./mvnw test.
- Informe de carga: con el resultado de ab.

Decisión de diseño: la entrega es un paquete completo. No basta con que el código funcione; hay que documentarlo para que otros puedan usarlo.

🕒 BLOQUE 5 - Criterios de aceptación

Para dar el proyecto por bueno, debe cumplir:

- Todos los endpoints funcionan según el diseño.
- Todos los tests pasan (BUILD SUCCESS).
- La seguridad funciona: 401 sin token, 403 sin permisos, 200 con permisos.
- La documentación está completa.
- La prueba de carga no produce errores (menos del 1% de fallos).
- El README está actualizado.